CS301 – IT Solution Architecture:
Solution Architecture Implementation Report

(15% of total marks)

1. Instructions

You are the solution architect documenting the architecture design. **Your audience is the sponsor's engineering and operations teams.**

This report summarizes the solution architecture for your system.

1. Your solution architecture needs to demonstrate your overall designs and implementations that fulfill the mandatory Features 1–4 as specified in the project requirements, in addition to your proposed X-factor feature. As compared to the presentation, your audience differs. **You need to convince the audience with a clear solution that they can understand your design and maintain the system.**
2. **(Report)** Limit your deliverable to a **maximum of 20 A4 pages, using Times New Roman, font size 12, with a minimum line spacing of 1.15.** Emphasize quality over quantity of content. The first page must contain only the project title and team member information, with no solution details. Page numbers must appear at the bottom right of each page. You must provide some **screenshots of the final app**.
3. **(GitHub)** All implementation code required to execute your application must be stored in GitHub. Any other codes/configuration subsequently added should also be in GitHub. When you create additional repos for your project, please adhere to the following format project-2025-26t2-g1-<t1-6>-<name of new repo>. (e.g., project-2025-26t2-g1-t1-backend) for easier identification. You must provide a **README.md** in the root of your GitHub folder for two sets of instructions. (1) For testing your current setup - please provide sufficient information for the evaluator to test your current setup. For example, URLs of your application, specific data values that can be used to test your solution. (2) For a new setup, please provide sufficient information if we wish to set up your solution in another region (e.g., Hong Kong).
4. **(App Testing)** You must use your assigned domain (e.g., *itsag1t1.com* for G1 T1) and use the AWS Singapore region as your primary deployment site. You must create default accounts for the CRM system: one administrator account and one agent account. The usernames must be **"admin@crm.com"** and **"agent1@crm.com"**. The corresponding passwords must be documented only in your report and must not be disclosed elsewhere. This is for evaluation purposes only; in real systems, credentials must not be documented in plaintext.
5. **(Submission)** Submit your deliverables via **eLearn->Assignment-> G{SectionNo} - Project Submissions** by the specified deadline. **PDF** your proposal deliverable document and name as **G1-{TeamNo}-Project-Final-Report.pdf (e.g., G1-T1-Project-Final-Report.pdf).** Late submission or failure to comply with the instructions will be penalized.

6. **(Teardown)** After the project presentation, you will receive explicit instructions (please wait for this) to terminate all AWS resources and will have one week to complete this task. **Failure to terminate AWS resources, especially those that incur costs, within the given timeframe will result in a grade penalty.**

2. Grading Criteria

1. [5%] Conformance to the instructions.
   [-] Missing, incomplete or late submission.
   [-] AWS resources not terminated completely and incur cost.
2. [5%] Your development on GitHub - commits and issues tracked.
   [-] Missing participation from most team members
3. [25%] Report quality and completeness of the use cases, diagrams or other notations.
   [-] Ambiguity or inconsistency among views or missing views
   [+] Clear descriptions, explanations, clearly readable figures and tables, scannable text
   [+] Good report presentation
4. [45%] Solution development for sponsor's functional and non-functional requirements (maintainability, availability, security, and performance.)
   [-] Missing designs or unable to implement specific sponsor requirements.
   [-] Wrong conceptual understanding of solution designs.
   [-] Alternative designs not compared or not well-supported by evidence or reputable references.
   [+] Bonus features or innovative designs implemented by the team provided basic features are already covered.
   [+] Quality designs tested with well-supported evidence (e.g., showing penetration test results to compare before and after earns more marks than simply locking down the solution. Showing performance results against a baseline earns more marks than merely configuring load balancing.)
   [+] Quality designs demonstrated by the team. Designs which are explicitly demonstrated carry more marks. For example, mentioning availability is handled by AWS service is counted but earns less marks than explicitly demonstrating the design such as terminating an instance while availability is preserved.
5. [20%] Solution proposed for the team's X-factor.
   [-] Design is not aligned with the project.
   [+] Design and implementation demonstrate clear potential impact for the sponsor.

3. Guidelines to the final report (can be customized)

- **GitHub Team name, team members (name and matriculation)**

- **Stakeholders**
  List the key business and IT stakeholders for this application. Refer to your proposal deliverable. Your list can differ from the proposal deliverable. Keep this short.

- **Key Use Cases**
  Document the **three** architectural significant requirements in the form of **use cases** for the system. The Admin and Agent ASRs must be derived from mandatory Features 1–4. Refer to your proposal deliverable. Your use cases write-up can differ from the proposal deliverable.

- **Proposed Budgets**
  You are required to compare your prototype solution **planned vs actual costs for prototype development**. Explain if the difference is more than 100 USD.

- **Architectural Decisions**
  Provide **three key architectural decisions** that inform the design of your solution, using the template provided below. Examples of such decisions include how the system is decomposed into microservices and the selection of a relational versus a non-relational database. For each architectural decision, discuss the alternative options considered and provide a clear justification for why the chosen approach is preferable for your proposed solution.

Architectural Decision - <short name for this decision.>	
ID	<any numbering format>
Issue	<what issue this decision is attempting to solve>
Architectural Decision	<short description of this decision>
Assumptions	<any assumptions made for this decision>
Alternatives	<what other possible solutions did you consider>
Justification	<explain why your alternatives are not appropriate>

- **Solution View**

  Provide an **overall view of the architecture design**. You can reuse your proposal deliverable. You can also show your key AWS routing configurations tables among the subnets/VPCs.

Solution View

  Provide the **integration endpoints** based on the following template. Your integration endpoints must include the external SFTP transaction source required in Feature 4.

  Integration Endpoints

Source System	Destination System	Protocol	Format	Communication Mode

  - **Source**: Sending system of the message.
  - **Destination**: Receiving system of the message. Middleware is also considered a system. If there is a middleware (e.g., MQ) between two systems, there should be a row for sending the system to MQ and another row from MQ to another system.
  - **Protocol**: The name of the protocol to send the message, e.g., HTTP, MQ. Use the highest layer, e.g., HTTP is a layer higher than TCP/IP.
  - **Format**: The format of the message, e.g., CSV, XML, JSON
  - **Communication Mode**: The mode of the message, e.g., synchronous or asynchronous.

  In addition, justify any other design(s) that addresses ease of maintainability (e.g., design patterns, architectural styles, microservices, multitenancy).

  (Bonus)

  - **(Infrastructure as Code)** Please provide templates for deploying your solution using code. This may include AWS CloudFormation templates or Terraform scripts designed to establish your foundational infrastructure in a new VPC/subnets and any additional shell/PowerShell scripts to further automate the process. If your scripts cannot automate the entire solution including the infrastructure, application and data, please highlight any manual steps to take.

  - **(CI/CD)** Demonstrate the methodology by which deliverables are developed, versioned, tested, and deployed across your simulated environment for this project.

- **Availability View**

  Provide a **view of the availability components** in the architecture. You can consider the availability design ideas covered in week 5. Note that components in this view must be already proposed in earlier solution view and do not introduce new components here. You can use the following template to list down your availability design.

Node	Redundancy	Clustering			Replication (if applicable)			
		Node Config.	Failure Detection	Failover	Repl. Type	Session State Storage	DB Repl. Config.	Repl. Mode

  - **Node**: Software or hardware or service instance
  - **Redundancy**: Horizontal or Vertical Scaling
  - **Node Configuration**: Active-Active or Active-Passive
  - **Failure Detection**: Ping or Heartbeat
  - **Failover**: Client, Load-balancer, DNS, Virtual IP
  - **Replication Type**: Session or DB
  - **Session State Storage**: Database, Client, Memory sessions
  - **Database Replication Config**: Master-Master or Master-Slave
  - **Replication Mode**: Synchronous or Asynchronous

  You can draw **sequence diagrams** to show how the system handles the loss of availability of specific architecture components.

Availability View

  (Bonus)

  - **(Redundancy, Detect Failure, Failover, State Management)** Provide a clear demonstration on how failure(s) do not disrupt the availability of your solution.

  - **(Eliminate Single Point of Failure)** Clearly illustrate using diagrams how at least one ASR is designed without any single point of failure at the architectural level.

- **Security View**

  Provide a **view of the security components** in the architecture. You can consider the security architecture design practices covered in week 3. Note that components in this view must be already proposed in earlier solution view and do not introduce new components here.
  Use the following template to list down the potential threats, vulnerabilities and controls.

No	Asset/Asset Group	Potential Threat/Vulnerability pair	Possible Mitigation Controls

  - **No**: Running sequence number. You should list the most significant ones - those with the highest risk.
  - **Asset/Asset Group**: Hardware, software, service, or data which are valuable to the project and organization.
  - **Potential Threat/Vulnerability pair**: The possible attack (threat) that can exploit the weakness (vulnerability) in your system. Be clear to state both the attack and weakness as a pair and how it affects the system (e.g., CIA – confidentiality, integrity, and availability).
  - **Possible Mitigation Controls**: The actions (controls) to be taken to mitigate the risk of the attack exploiting the vulnerability. State whether it is implemented or not.

  According to your solution view, you must also provide details of your key **AWS access control list** and **security groups**.

Security View

  (Bonus)
  - **(Penetration Testing)** Provide a clear demonstration with tools such as OWASP ZAP to show how vulnerabilities are effectively mitigated to minimize the attack surface. Please attempt to test remotely against your AWS resources and directly at your application resources. Your application can be vulnerable but protected by your AWS resources (e.g., WAF).

  - **(Defense in Depth)** Illustrate using diagrams, the design of at least one ASR that incorporates defense-in-depth strategies for mitigation.

- **Performance View**

  Provide a **view of the performance strategies** for the architecture. You can consider the performance heuristics covered in week 7. Note that components in this view must be already proposed in earlier solution view and do not introduce new components here.

  Use the following template to explain your strategies.

No	Description of the Strategy	Justification	Performance Testing (Optional)

  - **No**: Running sequence number. You should list the most significant ones - those potentially improve the performance the most.
  - **Description of the Strategy**: Detailed description of the strategy adopted. You can refer to the components in your solution view to describe your strategy — state whether it is implemented or not.
  - **Justification**: Explain why your strategy is better or more appropriate than other alternatives. You can refer to the architectural decisions if it is explained earlier.
  - **Performance Testing:** Optional. If your team conducts performance testing, show the baseline and the improved results.

  You can provide other measures implemented to improve performance.

Performance View

  (Bonus)

  - **(Load Testing)** Demonstrate application latency and response times under the required load using tools such as JMeter.

    The system must undergo load testing with 5 concurrent administrators for ASR (Admin's perspective) and 100 concurrent agents for ASR (Agent's perspective). The ramp-up period is set to 1 second, with a loop count of 3. Each agent manages an average of 100 clients (totaling at least 10,000 clients within the system), and each client has an average of 3 accounts and each account with 50 transactions (amounting to no fewer than 1,500,000 transactions overall).

    Performing stress tests to assess the system's performance under peak conditions will also be considered.

  - **(Reduce Bottlenecks)** Provide a clear illustration using diagrams, how at least one ASR is structured, detailing the identification and elimination of bottlenecks through the progression of events.